Criptografía, CSIRT y Malware

Manual académico completo de estudio · IC 2026 · UNLP

Guía elaborada a partir de las teorías de la cátedra de Introducción a la Ciberseguridad

Octubre de 2026

Contenido

1 Introducción y alcance 3
1.1 Ruta de estudio recomendada 4

2 PARTE I · Criptografía 5
2.1 Panorama: conceptos y relación con los incidentes 5
2.2 Codificación, ASCII y cifrado clásico 5
2.3 Hash y criptografía simétrica 8
2.4 Ejemplos adicionales resueltos de Criptografía 12

3 PARTE II · CSIRT 14
3.1 Fundamentos necesarios: CIA, amenazas e incidentes 14
3.2 Recursos de Internet: IP, DNS, WHOIS y RDAP 15
3.3 SMTP y análisis de encabezados de correo 20
3.4 Gestión de incidentes y equipos CSIRT 24
3.5 Casos adicionales resueltos de CSIRT 29

4 PARTE III · Análisis de Malware 31
4.1 Alcance y método de la teoría 31
4.2 Fundamentos y análisis estático 31
4.3 Análisis dinámico y persistencia 37
4.4 Comparación final: qué conclusiones permite cada evidencia 41

5 Integración y preparación para evaluaciones 43
5.1 Casos integradores resueltos 43
5.2 Caso A. Un correo sospechoso que anuncia una deuda 43
5.3 Caso B. Software descargado que parece un instalador 43
5.4 Caso D. Distinguir integridad, cifrado y autenticidad 44
5.5 Caso E. Dirección IP y clasificación de riesgo 44
5.6 Preguntas de autoevaluación con respuestas 45

6 Glosario de términos 51

7 Bibliografía, alcance y trazabilidad 53
7.1 Teorías de la cátedra empleadas 53
7.2 Referencias externas para profundizar (ampliaciones) 53
7.3 Comprobación de cobertura 54

1 Introducción y alcance

Esta **versión enfocada para el estudio de Criptografía, CSIRT y Análisis de Malware** está basada en las presentaciones de la materia *Introducción a la Ciberseguridad* (IC 2026, Facultad de Informática, UNLP). El manual prioriza la comprensión: **definiciones, fundamentos, ejemplos, comparaciones, interpretación de resultados y procedimientos**. Su organización responde a esos **tres ejes**, no al orden total de la cursada. No incluye capítulos de reversing, ensamblador ni exploiting.

Cómo distinguir fuentes y ampliaciones: las referencias **[CÁTEDRA: T04, p. n]** remiten a la página del PDF respectivo. Los apartados identificados como **«Ampliación»** son contexto añadido para explicar o actualizar ideas de clase; no son transcripciones literales. Las demostraciones y datos de registro reproducidos en las presentaciones son fotografías de una fecha y pueden variar al consultarlos hoy. Los ejemplos con dominios `.test` y direcciones reservadas para documentación son ficticios.

Documentos cubiertos en detalle:

Código	Teoría	Alcance en esta guía
T04	`04 Cripto parte 1.pdf` (30 pp.)	Representaciones, criptografía clásica, hash, simétrica, modos, ejemplos
T05	`05 CSIRT - Recursos de Internet.pdf` (20 pp.)	IP, RIR, WHOIS, RDAP, DNS, registración y contactos
T06	`06 encabezados_smtp.pdf` (17 pp.)	DNS/MX, sobre SMTP, encabezados, trazabilidad y suplantación
T07	`07 CSIRT - Gestión de incidentes.pdf` (26 pp.)	Vulnerabilidades, CSIRT, procesos, priorización, TLP y PGP
T08	`08 Analisis_de_malware.pdf` (57 pp.)	IOC, estático, PE, ofuscación, análisis dinámico, Registro, herramientas

La presentación **T03 (`03 Conceptos Básicos.pdf`)** se utiliza *solo como fundamento transversal* donde ayuda a definir confidencialidad, integridad, disponibilidad, amenazas y defensa en profundi-

dad. Se excluyen los temas generales ajenos a los tres bloques. Tampoco se incorporan los criterios administrativos de cursada de T01 ni la guía de writeups de T02.

1.1 Ruta de estudio recomendada

1. **Criptografía:** diferenciar codificación, cifrado y hash; reconocer criptografía clásica, cifrado simétrico y modos de operación.
2. **CSIRT:** comprender a quién corresponde un recurso de Internet, seguir un mensaje de correo, decidir la pertinencia y prioridad de un incidente y comunicar evidencia.
3. **Malware:** reconocer las familias y los IOC, interpretar un archivo PE sin ejecutarlo y explicar el valor y las limitaciones del análisis dinámico en un laboratorio.

Para evaluar tu comprensión, intentá resolver los casos integradores y las preguntas sin mirar las respuestas; consultá luego las referencias específicas.

2 PARTE I · Criptografía

2.1 Panorama: conceptos y relación con los incidentes

La criptografía estudia técnicas para proteger la información mediante transformaciones matemáticas que emplean algoritmos y, generalmente, claves. En el recorrido de la cátedra aparecen la **codificación** (no criptográfica), los métodos clásicos de **sustitución y transposición**, las **funciones hash** y el **cifrado moderno simétrico**. La presentación menciona también la existencia de criptografía asimétrica, pero el desarrollo de T04 se concentra en la simétrica. [CÁTEDRA: T04, pp. 5–16]

La primera parte de la presentación usa, como disparador, recortes de un **caso de posible compromiso de la cadena de suministro de software asociado con una dependencia de IA**. Su función didáctica es recordar que un sistema puede estar expuesto sin que su aplicación principal tenga una vulnerabilidad directa: las dependencias, su distribución y las credenciales de automatización también importan. **Ese episodio se presenta en diapositivas como noticia/captura y aquí no se toma como verificado independientemente.** [CÁTEDRA: T04, pp. 1–4]

Ampliación — buenas prácticas para dependencias: inventariar paquetes y versiones, fijar versiones e integridad cuando sea posible, proteger claves de CI/CD, controlar permisos y revisar alertas de seguridad. Ninguna de estas medidas sustituye el análisis de los comunicados del proveedor en un incidente concreto.

2.2 Codificación, ASCII y cifrado clásico

2.2.1 Codificar no es cifrar

Codificación (*encoding*) significa representar símbolos en otro sistema para facilitar almacenamiento, transmisión o interoperabilidad. No exige una clave secreta ni pretende ocultar datos. Las

diapositivas enumeran ASCII, representaciones numéricas, Unicode, EBCDIC, Morse, Braille, lengua de señas y Base64 como ejemplos de codificación/representación. [CÁTEDRA: T04, pp. 6–8]

Cifrado (*encryption*) transforma datos legibles (*plaintext*) en datos cifrados (*ciphertext*) mediante un **algoritmo y una clave**, con la intención de permitir su recuperación solo a quienes tengan la clave adecuada. **Descifrado** invierte la transformación con los parámetros correctos. No debe confundirse con **hashing**, que produce un resumen y no se descifra.

Proceso	¿Se revierte?	¿Requiere secreto?	Objetivo típico
Base64 / hex / ASCII	Sí, directamente	No	Representación de datos
AES bien utilizado	Sí, con clave correcta	Sí	Confidencialidad
SHA-256	No se «descifra»	No	Integridad e identificación de datos
Firma digital	Se verifica, no se «descifra» como mensaje	Clave privada para firmar	Autenticidad e integridad

2.2.2 Bases y ASCII

La base **2** usa dígitos 0 y 1; la **10** usa 0–9; la **16** usa 0–9 y A–F. El prefijo `0x` indica comúnmente notación hexadecimal. Un dígito hexadecimal equivale a 4 bits, por lo que un byte suele escribirse con **dos cifras hexadecimales**.

Ejemplo: representar la letra C. En ASCII, `C` tiene valor decimal **67**, hexadecimal **0x43** y binario **01000011**. La cadena `Ciberseg` se expresa con bytes hexadecimales `43 69 62 65 72 73 65 67`. ASCII estándar es una codificación de **7 bits**; UTF-8 conserva esos valores para ASCII e incorpora el resto de Unicode. [CÁTEDRA: T04, p. 8]

Conversión resuelta: `0x2D`. En decimal: `2 × 16 + 13 = 45`; en binario: `0010 1101`; en ASCII: `-`. Para reconstruir un texto a partir de hex hay que agrupar en bytes y aplicar la codificación esperada.

```
printf 'Hola' | xxd -g 1
printf '486f6c61' | xxd -r -p
printf 'Hola' | base64
printf 'SG9sYQ==' | base64 -d
```

Ampliación — Unicode. Unicode define puntos de código para representar caracteres de múltiples idiomas. UTF-8 y UTF-16 son formas de codificarlos en bytes. Un acento o un emoji puede ocupar más de un byte en UTF-8; en análisis de binarios no debe asumirse un byte por carácter para todo texto.

2.2.3 Cifrados clásicos: sustitución y transposición

Los **cifrados clásicos** son mecanismos históricos, generalmente manuales o electromecánicos, muy distintos en seguridad de las primitivas modernas. Se distinguen **sustitución**, que reemplaza símbolos, y **transposición**, que cambia su orden. La teoría menciona **ROT/César**, **escítala (scytale)**, **transposición columnar** y **máquina Enigma**. [CÁTEDRA: T04, pp. 11–14]

En el **cifrado César** se desplaza cada letra un número fijo de posiciones del alfabeto. Para letras `A=0, ..., Z=25`, la transformación puede expresarse como `C = (P + k) mod 26` y la inversa `P = (C − k) mod 26`. Con desplazamiento `k=3`, `HOLA` se convierte en `KROD`:

Entrada	Índice	+3 mod 26	Salida
H	7	10	K
O	14	17	R
L	11	14	O
A	0	3	D

En un desafío, `ROT13` significa desplazar 13 lugares y aplicar la operación dos veces recupera el original. **No es criptografía segura moderna**: probar los 26 desplazamientos es trivial.

Escítala. Un mensaje se enrollaba en torno a una vara y se escribía a lo largo de ella. Sin el grosor adecuado, el orden de lectura cambia; es una idea de **transposición**.

Transposición columnar. Se escribe el texto en una cuadrícula por filas y se leen columnas según un orden acordado. Las letras se conservan, pero se permutan. Un análisis de frecuencia puede revelar información porque no cambia cuántas veces aparece cada símbolo.

Enigma. Máquina electromecánica de sustitución polialfabética utilizada en la Segunda Guerra Mundial. Empleaba rotores y configuraciones variables. Ilustra cómo el diseño, la gestión de claves y la operación influyen en la seguridad, aunque su mecanismo es muy diferente al de AES.

Ampliación — principio de Kerckhoffs. La seguridad de un criptosistema moderno debe descansar en el secreto de la **clave**, no en ocultar el algoritmo. Conocer AES no permite descifrar un mensaje sin la clave adecuada.

2.3 Hash y criptografía simétrica

2.3.1 Qué es una función hash

Una **función hash criptográfica** transforma una entrada de longitud variable en un valor de tamaño fijo llamado *digest*. Las diapositivas remarcan cuatro propiedades: determinismo, resistencia práctica a invertir el resultado, cambio fuerte del digest ante modificaciones de entrada y cálculo eficiente. [CÁTEDRA: T04, pp. 16–18]

Ejemplo conceptual. Dos versiones de un texto solo difieren en la mayúscula de una letra. Sus valores SHA-256 serán completamente distintos. No existe una operación «SHA-256 decrypt»: comparar un candidato requiere volver a calcular su digest.

Las familias mencionadas son **MD5 (128 bits)**, **SHA-1 (160 bits)**, **SHA-2 (SHA-224/256/384/512)** y **SHA-3**. Para verificación moderna de integridad, SHA-256 es una opción habitual. MD5 y SHA-1 **no son adecuados para aplicaciones que dependen de resistencia a colisiones**, aunque se sigan viendo como identificadores heredados de archivos. [CÁTEDRA: T04, pp. 18–20]

```
printf %s 'Ciberseguridad' | sha256sum
printf %s 'CiberSeguridad' | sha256sum
sha256sum archivo.iso
md5sum archivo.iso
```

Atención a `echo`: agrega normalmente un **salto de línea** (`0x0a`) al final, por lo que `echo "abc" | sha256sum` y `printf %s abc | sha256sum` producen digest distintos. Las diapositivas muestran esa sutileza indirectamente al usar `echo` para ejemplos de MD5. [CÁTEDRA: T04, p. 18]

2.3.1.1 Propiedades de seguridad

Preimagen: dado un digest `h`, encontrar una entrada cualquiera cuyo hash sea `h` debería ser computacionalmente inviable para una función segura. **Segunda preimagen:** dado un mensaje, hallar otro distinto con el mismo digest. **Colisión:** hallar dos entradas diferentes que produzcan el mismo digest, sin fijar cuál de ellas. Las colisiones **deben existir matemáticamente** porque infinitas entradas se representan con salidas finitas; lo importante es que resulte impracticable encontrarlas intencionalmente.

La teoría exhibe dos imágenes distintas con el mismo MD5, demostrando que una colisión **no** implica que los archivos sean iguales. [CÁTEDRA: T04, p. 20]

2.3.1.2 Hash de contraseña frente a hash de archivo

Archivo: calcular SHA-256 para comparar una muestra conocida con su referencia es razonable; no prueba por sí solo que el origen sea confiable, pues un atacante podría alterar también el digest publicado.

Contraseña: aplicar SHA-256 o MD5 directamente no es suficiente. Como son rápidos, facilitan probar gran cantidad de candidatos. **Ampliación:** se emplean funciones de derivación *costosas y con sal* como Argon2id, scrypt, bcrypt o PBKDF2 según el entorno, nunca contraseñas en texto plano. La sal aleatoria evita hashes iguales para contraseñas iguales; el parámetro de costo dificulta ataques por fuerza bruta.

2.3.2 Criptografía simétrica y gestión de claves

En un **criptosistema simétrico**, emisor y receptor usan **la misma clave secreta** (o claves directamente derivadas de un secreto compartido) para cifrar y descifrar. El emisor selecciona o recibe la clave, cifra el texto y transmite el resultado; el receptor lo descifra con la clave correspondiente. [CÁTEDRA: T04, pp. 21–24]

Ventajas: alto rendimiento y apropiado para grandes cantidades de información. **Desafíos:** el secreto debe llegar a las partes por un canal seguro; compartir una clave entre muchas personas complica la revocación, atribución y escalabilidad. En las diapositivas aparecen **3DES, RC5, IDEA, AES y Blowfish** como ejemplos históricos o modernos. Hoy AES es la referencia más habitual de la lista para diseños nuevos. [CÁTEDRA: T04, p. 30]

2.3.3 Cifrados de bloque y modos de operación

Un **cifrador de bloque** recibe bloques de longitud fija y una clave. **AES** opera sobre bloques de **128 bits (16 bytes)** y permite claves de 128, 192 o 256 bits. El uso seguro en un archivo o flujo requiere elegir un **modo de operación** apropiado, además de gestionar IV, nonce, relleno y autenticación. La teoría enumera **ECB, CBC, CFB y OFB**. [CÁTEDRA: T04, pp. 24–28]

Modo	Concepto	Observación crucial
ECB	Cada bloque se cifra por separado con la misma clave	Bloques iguales originan cifrados iguales: revela patrones
CBC	Antes de cifrar se aplica XOR con el bloque cifrado anterior; el primero usa IV	Oculta igualdad de bloques, pero por sí solo no autentica
CFB	Realimentación del cifrado para operar por unidades menores	Requiere IV y tratamiento cuidadoso
OFB	Produce una secuencia cifrante desde el estado inicial	Reutilizar IV con la misma clave compromete seguridad

ECB paso a paso: si el texto contiene dos bloques iguales `P1 = P2`, entonces bajo la misma clave `E(K, P1) = E(K, P2)`; por lo tanto `C1 = C2`. En una imagen con regiones repetidas quedan patrones perceptibles. **No usar ECB para proteger contenido estructurado.**

CBC paso a paso: `C1 = E(K, P1 XOR IV)` y `C2 = E(K, P2 XOR C1)`. Para descifrar, `P1 = D(K, C1) XOR IV` y `P2 = D(K, C2) XOR C1`. El IV no debe confundirse con la clave. Para CBC, el IV debe ser impredecible al cifrar nuevos mensajes. [CÁTEDRA: T04, pp. 26–28]

Ampliación — cifrado autenticado. Actualmente suelen preferirse modos **AEAD** como **AES-GCM** o ChaCha20-Poly1305, que combinan confidencialidad con comprobación de integridad/autenticidad. El modo y los nonces deben implementarse siguiendo bibliotecas validadas; usar «AES» sin conocer el modo no basta para evaluar seguridad.

2.3.4 Cifrado de flujo

Un **cifrado de flujo** genera una secuencia pseudoaleatoria (*keystream*) que se combina, generalmente con XOR, con el texto plano. Es útil cuando los datos se procesan continuamente y no resulta cómodo esperar bloques completos. La teoría menciona **RC4** como ejemplo histórico, pero **RC4 está obsoleto para sistemas nuevos**. [CÁTEDRA: T04, p. 29]

Si `P = 10110010` y `K = 01100101`, entonces `C = P XOR K = 11010111`; al volver a aplicar el mismo `K`, recuperamos `P`. **El peligro central es reutilizar una secuencia cifrante**: la combinación de dos textos cifrados que reutilizan la misma secuencia revela información sobre los originales.

2.3.5 Contexto asimétrico (ampliación)

La teoría introduce la categoría **asimétrica** en su clasificación, pero el PDF T04 profundiza únicamente en la parte simétrica. Para ubicar el concepto: la criptografía asimétrica trabaja con una **clave pública** y una **clave privada** matemáticamente relacionadas. En un esquema de cifrado, la pública permite cifrar para un destinatario y la privada descifrar; en firmas digitales, la privada firma y la pública permite verificar. **RSA, ECDSA, Ed25519 y los protocolos de acuerdo de claves** tienen propósitos diferentes: no conviene confundir cifrado, firma y establecimiento de claves. Este apartado es **contexto complementario, no unidad desarrollada en T04**.

Preguntas de autocontrol de criptografía. ¿Por qué Base64 no equivale a cifrado? ¿Por qué un digest no puede descifrarse? ¿En qué se diferencian colisión y segunda preimagen? ¿Por qué no se debería usar MD5 para verificar una firma? ¿Qué patrón revela ECB? ¿Por qué un cifrado rápido no es ideal para almacenar contraseñas? ¿Qué significa compartir una clave secreta entre diez integrantes de un equipo?

2.4 Ejemplos adicionales resueltos de Criptografía

2.4.1 Ejemplo 1. De texto a bytes y de bytes a Base64

Partimos del mensaje `IC`. Con ASCII, `I` corresponde a decimal 73 / hexadecimal `49`, y `C` a 67 / `43`. La secuencia de bytes es `49 43`. Base64 divide el flujo de bits en grupos de seis y lo representa usando su alfabeto; para estos dos bytes el resultado es `SUM=`. La transformación inversa reproduce los mismos bytes: **no hay secreto** en el mecanismo.

```
printf 'IC' | xxd -g 1
printf 'IC' | base64
printf 'SUM=' | base64 -d
```

2.4.2 Ejemplo 2. Comparación que falla por un byte invisible

Una organización calcula SHA-256 de una contraseña de prueba con `echo` y con `printf`. Con `echo`, salvo la opción adecuada, se agrega `0x0a` (*newline*). Aunque la palabra visible sea la misma, **no son las mismas entradas**, de modo que sus hashes deben ser distintos. Para verificar archivos también importa el contenido exacto, no su nombre ni lo que parezca mostrar un editor.

```
echo 'hola' | sha256sum
printf %s 'hola' | sha256sum
```

2.4.3 Ejemplo 3. Dos bloques iguales con ECB y CBC

Supongamos que los primeros 16 bytes y los siguientes 16 bytes de un archivo son idénticos. **ECB**, con igual clave, producirá bloques de salida idénticos. En **CBC**, el segundo bloque recibe antes del cifrado un XOR con el bloque cifrado anterior; por ello normal-

mente produce una salida diferente. Esto explica el dibujo comparativo Original / ECB / CBC de la teoría, donde ECB conserva patrones visuales. [CÁTEDRA: T04, pp. 25–28]

Precisión didáctica: CBC necesita un IV apropiado y protección de integridad adicional; para diseños modernos suele preferirse un modo autenticado como GCM. No se debe elegir ECB para ocultar información estructurada.

2.4.4 Ejemplo 4. ¿Qué garantiza cada control?

Objetivo	Técnica útil	Lo que no demuestra por sí sola
Ocultar un archivo	Cifrado con clave secreta	Que no haya sido alterado
Detectar que un archivo cambió	Comparación SHA-256 con referencia confiable	La identidad del autor del archivo
Verificar quién firmó un dato	Firma verificada con clave pública confiable	Que el archivo no tenga malware
Representar binarios como texto	Base64	Confidencialidad

Ampliación: las firmas digitales y el cifrado autenticado permiten combinar propiedades, pero hay que separar **confidencialidad**, **integridad** y **autenticidad** al responder preguntas teóricas.

3 PARTE II · CSIRT

3.1 Fundamentos necesarios: CIA, amenazas e incidentes

La seguridad de la información protege **confidencialidad** (acceso limitado a autorizados), **integridad** (creación/modificación autorizada) y **disponibilidad** (acceso oportuno de usuarios autorizados), conocidos como la **tríada CIA**. La cátedra utiliza esa tríada como punto de partida del análisis de incidentes. [CÁTEDRA: T03, pp. 2–4; T07, p. 2]

- Una **vulnerabilidad** es una debilidad en sistema, comunicación, infraestructura o personas, explotable por una amenaza.
- Una **amenaza** es un actor o suceso que puede afectar la información al aprovechar condiciones vulnerables; puede ser humana maliciosa, humana no intencional o natural.
- Un **riesgo** combina probabilidad de ocurrencia e impacto potencial; la expresión `amenaza × vulnerabilidad × impacto` de las teorías es un **modelo conceptual**, no una ecuación universal con unidades fijas.
- Un **incidente** es un evento adverso que **se concretó** y compromete o pone en peligro la seguridad. Puede consistir en ransomware, robo de credenciales, fuga de datos, defacement o acceso no autorizado. [CÁTEDRA: T03, pp. 5–8; T07, pp. 3–6]

Ejemplo: un servidor tiene una contraseña débil (vulnerabilidad); se realizan intentos de acceso maliciosos (amenaza actuando); antes de verificar ingreso exitoso puede haber un evento o intento; cuando se confirma un acceso no autorizado existe evidencia de incidente. La **prioridad** depende además de los datos/servicios afectados.

Ampliación — defensa en profundidad: combinar segmentación, MFA, controles de endpoint, monitoreo, respaldos y procedimientos disminuye el daño cuando falla una barrera. La noción aparece en T03, p. 18, pero el desarrollo operativo siguiente proviene principalmente de T05–T07.

3.2 Recursos de Internet: IP, DNS, WHOIS y RDAP

3.2.1 Los recursos y sus responsables

Ante un incidente originado desde una IP o relacionado con un dominio, el analista necesita averiguar **qué entidad administra ese recurso** y cómo contactar a las personas adecuadas. Los recursos abordados en la presentación son los **nombres de dominio**, las **direcciones IPv4** y las **direcciones IPv6**. Una IP permite encaminar paquetes; un nombre DNS asocia identificadores legibles a diversos tipos de registros; **no** son equivalentes a identidad de la persona que realiza una conexión. [CÁTEDRA: T05, pp. 1–2]

IPv4: direcciones de 32 bits, normalmente en cuatro octetos, ejemplo `192.0.2.10`. **IPv6:** direcciones de 128 bits, representadas como grupos hexadecimales, ejemplo `2001:db8::10`. Esas dos direcciones de ejemplo pertenecen a rangos reservados para documentación y no identifican un servidor real. En formato CIDR, `/24` para IPv4 designa 24 bits de prefijo de red y deja ocho para direcciones; `/64` para IPv6 marca 64 bits de prefijo.

Ampliación — límites de la atribución. Una misma IP pública puede representar un router con NAT, un proxy, una VPN, una plataforma de nube o numerosos clientes. Los datos de asignación vinculan direcciones a **organizaciones**, no identifican automáticamente al autor de un ataque. Para contextualizar un incidente pueden hacer falta puertos, tiempos exactos, logs, correlación y autorización legal.

3.2.2 De IANA a los registros regionales

Los **RIR** (*Regional Internet Registries*) administran la distribución de recursos numéricos de Internet por regiones y conservan registros públicos de asignación. La teoría enumera cinco: [CÁTEDRA: T05, pp. 3–5]

Registro	Región de servicio principal
ARIN	Estados Unidos, Canadá y partes del Caribe / Atlántico Norte
APNIC	Asia y Oceanía
LACNIC	América Latina y regiones del Caribe
RIPE NCC	Europa, Oriente Medio y Asia Central
AFRINIC	África y algunas regiones del océano Índico

IANA coordina recursos globales como el espacio numérico y la raíz DNS; los RIR administran asignaciones y/o registros en sus regiones. Una organización que solicita direcciones provee información administrativa y de contacto para que sea posible ubicar responsables, incluidos contactos de **abuso** (*abuse*) y equipos CSIRT.

Ejemplo de pertenencia. Un reporte informa tráfico problemático desde `163.10.5.66`. La teoría enseña que el bloque `163.10.0.0/16` fue consultado mediante **LACNIC** y corresponde al registro histórico mostrado de la Universidad Nacional de La Plata. El resultado es evidencia de **asignación**, no prueba de quién realizó una acción. [CÁTEDRA: T05, pp. 9–11]

3.2.3 WHOIS y RDAP

WHOIS es un protocolo y familia de bases de consulta históricas. Sus respuestas suelen ser texto libre, sin un esquema único, por lo que analizarlas automáticamente resulta difícil. **RDAP** (*Registration Data Access Protocol*) es su sucesor estandarizado, usa **HTTP(S)** y documentos **JSON**, facilitando automatización y consultas estructuradas. [CÁTEDRA: T05, pp. 6–7 y 12–13]

Dimensión	WHOIS	RDAP
Formato de salida	Texto, variable según servidor	JSON con estructura especificada
Uso común	Consulta manual e histórica	Integración y procesamiento automático
Transporte habitual	TCP/43	HTTPS

Dimensión	WHOIS	RDAP
Información disponible	Recursos, organización, contactos	Recursos, eventos, entidades, contactos
Límite común	Respuestas ambiguas o remisiones	Posibles omisiones/redacciones por política de privacidad

Ejemplo de WHOIS.

```
whois 190.15.133.5
whois 163.10.5.66
whois -h whois.lacnic.net 163.10.5.66
```

En el caso de `190.15.133.5`, T05 presenta un registro con `inetnum: 190.15.128.0/20` y `owner: CEDIA`. Entre los campos que se investigan aparecen `inetnum` (rango), `owner` (titular registrado), `country`, contacto administrativo, técnico y de abuso, fechas de creación y modificación, y servidores asociados. [CÁTEDRA: T05, pp. 7–8]

3.2.3.1 Los bloques legacy / ERX

Una dificultad histórica es que **algunas redes se registraron antes de los actuales RIR**. Al consultar un servidor no correspondiente, se puede recibir un registro general como `ERX-NETBLOCK` en vez de la entidad final. La presentación muestra este fenómeno con `163.10.5.66`: una respuesta de APNIC para un bloque histórico no es el dato operativo buscado; consultar directamente a **LACNIC** devuelve el registro más específico. [CÁTEDRA: T05, pp. 9–11]

Procedimiento de interpretación: (1) identificar el RIR aparente; (2) determinar si la respuesta es un registro genérico de transferencia; (3) seguir la remisión al registro competente; (4) buscar el prefijo específico; (5) localizar el contacto de abuso y conservar fecha, hora y fuente de la consulta.

3.2.3.2 Anatomía de una respuesta RDAP

El cliente `icann-rdap` citado en el material está disponible en el repositorio oficial de ICANN. La teoría ejecuta `./rdap 190.15.133.5` y examina un JSON que contiene elementos como `rdapConformance`, `objectClassName`, `handle`, `links`, `events`, `status` y `entities`. [CÁTEDRA: T05, pp. 12–13]

Ejemplo esquemático (no salida literal de un servidor):

```
{
  "objectClassName": "ip network",
  "handle": "192.0.2.0/24",
  "status": ["active"],
  "events": [{"eventAction": "registration",
              "eventDate": "2024-01-01T00:00:00Z"}],
  "entities": [{"handle": "ORG-EXAMPLE"}]
}
```

`handle` identifica el objeto registrado, `events` guarda eventos fechados, `links` relaciona recursos y `entities` representa organizaciones o contactos. El formato puede incorporar `vcardArray` para datos estructurados de identidad. **La ausencia de un dato no significa que el recurso no tenga responsable**: puede estar reservado, redactado, delegado o no expuesto públicamente.

Ampliación — estándar. Las consultas RDAP se describen en la familia RFC 7480–7484. Una URL de consulta válida tiene estructura similar a `https://servidor-rdap/rdap/ip/192.0.2.0`. Su servidor efectivo depende del registro competente; no hay que asumir que cualquier URL de ejemplo responde.

3.2.4 DNS: jerarquía, nombres y tipos de registro

El **DNS** (*Domain Name System*) es tanto una **base de datos distribuida y jerárquica** como un **protocolo de consulta**. La jerarquía mencionada en la teoría comienza en la **raíz** `.`, continúa con los **TLD** y se extiende a dominios, subdominios y hosts. [CÁTEDRA: T05, pp. 14–15]

Ejemplo de jerarquía: en `mail.unlp.edu.ar.`, el punto final representa la raíz, `ar` es el ccTLD, `edu.ar` está debajo, `unlp.edu.ar` es el dominio y `mail` identifica un nombre específico dentro de él. El nombre **no prueba necesariamente** que exista una sola máquina o un solo servicio asociado.

Tipos de TLD: los **gTLD**, por ejemplo `.com`, `.net` y `.org`; los **ccTLD**, por ejemplo `.ar`, `.cl` y `.es`; y `.arpa`, reservado para infraestructura técnica como resolución inversa. La teoría menciona expresamente que el árbol DNS se gestiona de forma descentralizada.

Ampliación — registros de interés:

Tipo DNS	Qué representa	Uso típico de análisis
A	IPv4 asociada a nombre	Localizar infraestructura
AAAA	IPv6 asociada a nombre	Localizar infraestructura IPv6
MX	Servidores que reciben correo y prioridades	Rastrear ruta esperable del correo
NS	Servidores autoritativos de una zona	Identificar delegación
TXT	Texto arbitrario	Consultar SPF, verificaciones y otras políticas
PTR	Asociación inversa IP a nombre	Contextualizar servidores; no autentica por sí solo
CNAME	Alias de un nombre a otro	Seguir indireccionamientos

```
host -t a example.org
host -t aaaa example.org
host -t mx example.org
host -t ns example.org
dig example.org TXT
dig -x 192.0.2.10
```

3.2.5 Registry, registrar y registrant

El **registry** conserva el registro oficial de un TLD. Un **registrar** comercializa la registración del nombre de dominio y mantiene relaciones con sus registrantes. El **registrant** es la persona u organización a nombre de la cual se inscribe el dominio. La misma entidad puede desempeñar más de un papel. T05 usa a NIC Argentina y al espacio `.ar` como ejemplo de gestión de nombres. [CÁTEDRA: T05, pp. 16–18]

Ejemplo operativo: ante un dominio usado para una página fraudulenta, una consulta WHOIS/RDAP del dominio puede mostrar **registrar**, fechas de alta/vencimiento, servidores de nombre y correo de abuso. Los datos del registrante pueden estar **redactados por privacidad**. Ello no debe confundirse con ocultamiento malicioso; es una práctica habitual de protección de información personal.

3.2.6 Para qué utiliza un CSIRT esta información

Los equipos de respuesta emplean registros de IP y dominio para **reportar abusos, identificar operadores de servicios y coordinar mitigación**. Las **agencias de investigación** pueden obtener datos no públicos siguiendo la normativa y el proceso legal aplicables. [CÁTEDRA: T05, pp. 19–20]

Caso resuelto. Se recibe un log de intentos de autenticación desde una IP. Primero se verifica que la dirección sea pública y que el timestamp incluya zona horaria; después se consulta RDAP para averiguar quién administra el rango; luego se buscan los contactos de abuso y se notifica con el mínimo registro necesario. Si la IP pertenece a un proveedor de nube, se puede requerir identificación adicional del recurso en el momento del evento.

3.3 SMTP y análisis de encabezados de correo

3.3.1 SMTP, DNS y otros protocolos

SMTP (*Simple Mail Transfer Protocol*) se utiliza para transferir correo. La presentación muestra el envío entre servidores típicamente por **TCP/25** y la lectura del buzón con **POP o IMAP**; estas últimas tecnologías no realizan la misma función que SMTP. El DNS interviene por medio de los registros **MX**, que anuncian servidores destinatarios de un dominio. [CÁTEDRA: T06, pp. 1–3]

Ejemplo de recorrido: cliente del emisor → servidor de envío → servidores intermedios → servidor receptor → buzón del destinatario → lector mediante IMAP/POP. En operaciones modernas también son frecuentes SMTP de envío autenticado y conexiones cifradas; no hay que confundir los puertos de **sumisión** del cliente con el puerto 25 de transferencia entre servidores.

```
host -t mx info.unlp.edu.ar
dig info.unlp.edu.ar MX
```

El ejemplo de clase devuelve distintos servidores MX de `info.unlp.edu.ar`, con números de preferencia **10, 20 y 30**. En general, un menor número representa mayor preferencia; los MX no acreditan que un mensaje particular efectivamente haya atravesado uno de esos servidores. [CÁTEDRA: T06, p. 3]

3.3.2 Tres partes conceptuales de un mensaje

1. Envelope / sobre SMTP: conjunto de directivas de transporte, especialmente `MAIL FROM` (remitente de sobre, usado en notificaciones de fallo) y `RCPT TO` (destinatarios reales de entrega). **No equivale** al encabezado visible `From:`.

2. Encabezados: líneas informativas bajo el formato de mensajes de Internet (RFC 5322), incluyendo `From:`, `To:`, `Date:`, `Subject:`, `Reply-To:`, `Received:`, `Message-ID:` y muchas otras.

3. Cuerpo: contenido del mensaje, separado de los encabezados mediante una línea vacía. MIME permite adjuntos, multipartes y codificaciones, no desarrolladas en profundidad en esta presentación. [CÁTEDRA: T06, pp. 4–6]

Comparación práctica:

```
MAIL FROM:<bounce@ejemplo.test>   <- SMTP: sobre
RCPT TO:<victima@ejemplo.test>    <- SMTP: sobre
DATA
From: "Soporte" <soporte@ejemplo.test>  <- cabecera
To: victima@ejemplo.test
Reply-To: respuesta@ejemplo.test
Subject: Verificar su cuenta

Texto del mensaje                   <- cuerpo
.
```

Las direcciones son ilustrativas. El remitente de sobre y el `From:` pueden diferir **sin que necesariamente exista un ataque**; ocurre, por ejemplo, en plataformas que separan envíos y rebotes. Es sospechoso cuando la diferencia contradice el servicio esperado o coincide con otros indicadores de fraude.

3.3.3 Una sesión SMTP y sus códigos

La presentación muestra una sesión manual usando `telnet` con comandos `HELO`, `MAIL FROM`, `RCPT TO`, `DATA` y `QUIT`. Las respuestas del servidor suelen comenzar con códigos de tres cifras: **220** bienvenida, **250** operación aceptada, **354** comenzar a enviar el contenido, **221** cierre; **5xx** indican rechazos permanentes y **4xx** errores temporales. [CÁTEDRA: T06, pp. 11–12]

Ejemplo esquemático de sesión en un entorno de pruebas autorizado:

```
S: 220 mail.lab.test ESMTP
C: EHLO cliente.lab.test
S: 250-mail.lab.test
S: 250 OK
C: MAIL FROM:<usuario@lab.test>
S: 250 OK
C: RCPT TO:<receptor@lab.test>
S: 250 OK
C: DATA
S: 354 End data with <CRLF>.<CRLF>
C: From: usuario@lab.test
C: To: receptor@lab.test
C: Subject: Mensaje de prueba
C:
C: Hola.
C: .
S: 250 Queued
C: QUIT
S: 221 Bye
```

Un servidor abierto en Internet puede rechazar el envío por políticas de retransmisión, autenticación, reputación o cifrado. La conexión de ejemplo **no** indica que sea correcto intentar enviar mensajes arbitrarios por servidores ajenos.

3.3.4 Lectura forense de encabezados

Un encabezado **Received** suele ser agregado por cada servidor que acepta el mensaje. Por ello, **la secuencia generalmente se lee desde abajo hacia arriba** para reconstruir los saltos más antiguos a los más recientes. Sin embargo, el atacante puede insertar falsas líneas Received antes de entregar el mensaje: es importante identificar el **primer servidor bajo control o confianza del receptor** y verificar sus logs cuando están disponibles. [CÁTEDRA: T06, pp. 7–9 y 15–17]

```
Received: from relay.lab.test (relay.lab.test [192.0.2.30])
  by inbox.lab.test with ESMTP; Tue, 13 Oct 2026 10:05:00 -0300
Received: from pc.lab.test ([192.0.2.20])
  by relay.lab.test with ESMTP; Tue, 13 Oct 2026 10:04:40 -0300
From: "Mesa de ayuda" <ayuda@lab.test>
Reply-To: pagos@lab.test
Date: Tue, 13 Oct 2026 10:04:30 -0300
Subject: Actualizar datos
```

Interpretación resuelta: `relay.lab.test` entregó a `inbox.lab.test` a las 10:05; el mensaje aparentemente había ingresado a `relay.lab.test` desde `192.0.2.20` a las 10:04:40. El campo `Reply-To` hace que una respuesta vaya a otra casilla; por sí solo no prueba fraude, pero exige contexto. La primera IP observable **no siempre será la PC física del remitente**: puede ser un relay o servicio de envío.

Qué extraer sistemáticamente: IPs y nombres de host de los saltos verificables; fechas y zonas horarias; transportes y uso de TLS; `From`, `Reply-To`, `Return-Path`, `Message-ID`, resultados de autenticación; y diferencias entre lo esperado por DNS y lo observado. Diferenciar IP privadas del RFC 1918 (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) de direcciones públicas evita atribuciones incorrectas. [CÁTEDRA: T06, pp. 2 y 7–10]

3.3.5 Suplantación del remitente: sobre frente a cabecera

La demostración de clase muestra que **envelope `MAIL FROM` y encabezado `From:` son independientes**. Es técnicamente posible que el transporte declare un remitente y el encabezado otro, dando lugar a mensajes que **parezcan** venir de una entidad distinta. También se puede manipular `Reply-To` para dirigir la respuesta a otro destino. [CÁTEDRA: T06, pp. 10–14]

Ampliación — SPF, DKIM y DMARC:

- **SPF:** el dueño de un dominio publica qué servidores pueden enviar correo en su nombre según la identidad comprobada por SPF, normalmente la de sobre.
- **DKIM:** firma criptográficamente ciertas cabeceras y cuerpo, y permite verificar que el contenido no fue modificado bajo el dominio firmante.

- **DMARC:** define reglas de alineación de identidades y políticas frente a fallos de SPF/DKIM.

Estas tecnologías aumentan la protección, pero ningún resultado aislado permite asegurar que un mensaje es inocuo: una cuenta legítima puede haber sido comprometida. La teoría de SMTP enfatiza **leer las cabeceras originales**, sin confiar ciegamente en herramientas automáticas como **MXToolbox Email Headers** o **Google Admin Toolbox Messageheader**. [CÁTEDRA: T06, p. 16]

3.3.6 Método para investigar un correo sospechoso

1. Guardar el **mensaje original con encabezados completos**; una captura del cuerpo no basta.
2. Anotar fecha, hora y zona horaria; correlacionar diferencias razonables entre saltos.
3. Revisar `From:`, `Reply-To:`, `Return-Path:` y destinatarios reales cuando puedan recuperarse del servidor.
4. Leer los `Received:` de abajo arriba, distinguiendo saltos que realmente controla una entidad confiable.
5. Consultar DNS MX, A/AAAA, PTR y, si hay registros disponibles, SPF/DKIM/DMARC.
6. Correlacionar IP, dominio, URLs y archivos adjuntos con evidencia del incidente; **no abrir vínculos o adjuntos peligrosos**.
7. Formular una conclusión con nivel de confianza y datos que podrían faltar.

Ejercicio: una cabecera `From: administrador@universidad.test` convive con `Reply-To: cobros@otro.test`. ¿Es forzosamente phishing? **Respuesta:** no. Es un indicador que debe cruzarse con contexto, autenticación, ruta y legitimidad del pedido.

3.4 Gestión de incidentes y equipos CSIRT

3.4.1 Qué es un incidente y cuándo interviene un CSIRT

La teoría define un **incidente de seguridad** como un evento adverso que compromete la seguridad de la información. Incluye ejemplos: **defacement** (modificación no autorizada de un sitio),

ransomware, robo de credenciales, fuga de datos, compromiso de hosts o servicios y accesos no autorizados. [CÁTEDRA: T07, pp. 2–6]

Un **CSIRT** (*Computer Security Incident Response Team*) o **CERT** (*Computer Emergency Response Team*) coordina el tratamiento de incidentes de su **comunidad objetivo** —por lo común, la organización a la que sirve—. También atiende impactos causados **a terceros desde recursos administrados por esa comunidad**. La presentación indica otras siglas relacionadas, como CSIRC, CIRC, CIRT, IHT, IRC, IRT, SERT y SIRT; la denominación varía, pero la función central es la respuesta a incidentes. [CÁTEDRA: T07, pp. 7 y 20–21]

Ejemplo de pertinencia. Una cuenta universitaria recibe phishing: el incidente afecta a un miembro de la comunidad y puede corresponder a su CSIRT. Si una máquina de su red está enviando spam a terceros, aunque la víctima esté afuera, **el recurso de origen** pertenece a su ámbito y el CSIRT debe actuar. Si un tercero reporta actividad desde otra red sin relación, puede corresponder **derivar** la notificación en lugar de intervenir técnicamente.

3.4.2 Recepción, pertinencia, clasificación, gestión

La secuencia que desarrolla T07 es **recepción → evaluación de pertinencia → clasificación y priorización → gestión y notificaciones → coordinación**. [CÁTEDRA: T07, pp. 8–10]

Recepción: los reportes pueden llegar de usuarios internos, usuarios externos, otros CSIRT o mecanismos de observación como monitoreo de red y escaneo de vulnerabilidades. El primer registro debe conservar informante, evidencia, recurso, tiempo y estado.

Pertinencia: una alerta no es necesariamente un incidente. Se pregunta (a) si el hecho afecta un activo o usuario de la comunidad y (b) si se originó desde sus recursos. Los ejemplos visuales de la presentación muestran reportes de phishing cuyo análisis cambia según la **evidencia completa de cabeceras**, no solo el aspecto visual del correo. [CÁTEDRA: T07, pp. 11–12]

Clasificación: se identifica el tipo de incidente: phishing, ransomware, escaneo, fuerza bruta, host comprometido, base de datos expuesta, denegación de servicio o vulnerabilidad crítica, entre otros. La diapositiva contiene una lista más amplia de nombres de categorías y debilidades que suelen aparecer en plataformas de gestión. [CÁTEDRA: T07, p. 14]

Priorización: se estima criticidad según naturaleza del hecho, activos implicados, efecto en tareas críticas y daño a terceros. No es suficiente decir «spam = bajo» en todos los casos: un equipo que participa en ataques masivos o que contiene datos sensibles puede requerir atención urgente. [CÁTEDRA: T07, pp. 13–17]

Situación	Evaluación orientativa	Por qué
Un único equipo de laboratorio envía spam	Menor prioridad relativa	Alcance aparentemente limitado; requiere investigar
Base de datos pública con historiales sensibles	Muy alta	Riesgo grave de confidencialidad
Red de PCs comprometidas lanzando DDoS	Alta	Impacto a terceros, propagación, reputación
Un servicio crítico muestra acceso no autorizado	Muy alta	Posible interrupción y compromiso de procesos esenciales

Estas prioridades son ejemplos, no una matriz oficial de severidad. Siempre se ajustan a la organización y al contexto técnico.

3.4.3 Notificar correctamente

La **notificación** debe llegar a quien administra el recurso afectado y aportar información que permita **verificar e investigar**. La teoría insiste en incluir **evidencia técnica con timestamps y zona horaria**. [CÁTEDRA: T07, pp. 18–19]

Modelo didáctico de notificación:

> Asunto: Actividad sospechosa observada desde 192.0.2.42
> Detectamos intentos repetidos de acceso no autorizado a un servicio bajo nuestra administración el 15/10/2026 entre 10:00 y 10:12 UTC−3, con origen visible 192.0.2.42. Adjuntamos registros de fecha, IP, destino y tipo de fallo.

> Solicitamos verificar si el equipo o servicio bajo su responsabilidad fue comprometido. Podemos proporcionar evidencia adicional a través de un canal seguro.

Buenas prácticas: determinar contacto a través de WHOIS/RDAP o canales institucionales; evitar acusaciones categóricas sobre personas; compartir solo la evidencia necesaria; proteger datos sensibles y registrar comunicaciones y decisiones.

3.4.4 Servicios reactivos, proactivos y de gestión

El CSIRT no se limita a reaccionar. La presentación distingue tres grupos: [CÁTEDRA: T07, p. 22]

- **Reactivos:** alertas, gestión de incidentes y análisis forense.
- **Proactivos:** evaluaciones y auditorías de seguridad, herramientas de protección, detección de intrusiones.
- **Gestión de calidad:** continuidad operativa, recuperación ante desastres, asesoría, concientización y capacitación.

Ampliación — ciclo de respuesta. En NIST SP 800-61 rev. 3, la respuesta a incidentes se integra a la gestión global del riesgo y a las funciones del NIST CSF 2.0. Resulta útil asociar el trabajo del CSIRT con **preparar, detectar, analizar, contener, erradicar, recuperar y aprender**, recordando que la organización y las funciones formales de NIST han evolucionado frente a las listas pedagógicas anteriores.

3.4.5 Prácticas de confidencialidad, coordinación y evidencia

Un CSIRT necesita manejar con cuidado la información de incidentes: una muestra o log puede contener nombres de empleados, claves, tokens, información de víctimas y detalles que empeoran la situación si se difunden. La teoría considera explícitamente la protección de datos y el intercambio con terceros cuando es necesario. [CÁTEDRA: T07, pp. 23–26]

Ampliación — conservación forense. Preservar **cadena de custodia** significa documentar de dónde sale una evidencia, quién accede, cuándo se hace una copia y cómo se verifica su in-

tegridad, por ejemplo mediante SHA-256. Registrar zona horaria y diferenciar evento, recolección e informe evita atribuir una secuencia falsa.

Ejercicio integrador: un CSIRT recibe «El sitio `ejemplo.test` manda phishing». Falta saber si el dominio pertenece a su comunidad, si el mensaje es auténtico, si el sitio está realmente comprometido, cuál es el contacto correcto y qué prioridad asignar. Lo adecuado es solicitar cabeceras y URLs, confirmar pertinencia, recabar evidencia, clasificar, avisar a responsables y documentar acciones, no dar por probado el compromiso solo por una denuncia.

3.4.5.1 Intercambio de información: TLP y PGP

La cátedra presenta el **Traffic Light Protocol (TLP)** como un método para indicar **hasta dónde se permite compartir una información sensible** durante la coordinación entre CSIRT, incluyendo una referencia visual al caso WannaCry. **TLP no cifra los datos**: establece una regla de difusión. [CÁTEDRA: T07, pp. 23–25]

Ampliación — TLP 2.0: `TLP:RED` restringe la comunicación a receptores directos; `TLP:AMBER` permite compartir conforme al criterio de «necesidad de saber» dentro de la organización y sus clientes cuando corresponda; `TLP:AMBER+STRICT` limita a la organización; `TLP:GREEN` permite compartir en una comunidad; `TLP:CLEAR` no impone límites de TLP. El detalle de aplicación se rige por la versión vigente del estándar FIRST y los acuerdos de cada intercambio.

PGP (*Pretty Good Privacy*) es un conjunto de tecnologías de criptografía de clave pública usado para **cifrar y/o firmar** información. T07 lo cita como alternativa a una reunión presencial para intercambiar reportes de pentest, forensia e información confidencial. Para un informe, cifrar para los destinatarios autorizados limita quién puede leerlo; firmarlo ayuda a verificar autenticidad e integridad. Una **firma no equivale a confidencialidad** y **el cifrado no identifica por sí solo al autor**. [CÁTEDRA: T07, p. 26]

3.5 Casos adicionales resueltos de CSIRT

3.5.1 Caso 1. ¿A qué entidad enviar el reporte?

Dato: los logs del laboratorio registran tráfico no deseado desde `163.10.5.66`. La diapositiva de WHOIS advierte que, para ese rango histórico, una consulta por defecto puede terminar en un registro ERX genérico, y muestra la consulta explícita al servidor de LACNIC que identifica el bloque `163.10.0.0/16` asignado a la **Universidad Nacional de La Plata** según la salida capturada en clase. La asignación **es un dato administrativo**, no una prueba de que la universidad haya iniciado una agresión deliberada. [CÁTEDRA: T05, pp. 9–11]

```
whois 163.10.5.66 -h whois.lacnic.net
```

Razonamiento: (1) comprobar la IP y hora de la evidencia; (2) identificar el bloque concreto; (3) ubicar contacto de abuso y correo institucional apropiado; (4) remitir solo información necesaria con marcas temporales y zona horaria; (5) evitar afirmar la identidad del usuario final que actuó detrás de esa IP.

3.5.2 Caso 2. Un correo con tres identidades distintas

```
MAIL FROM:<rebotes@plataforma.test>  # sobre SMTP
From: "Mesa de ayuda" <servicio@institucion.test>
Reply-To: cobros@otro-dominio.test
```

Pregunta: ¿qué destinatario recibe un error de entrega y a dónde responde el usuario? El **rebote** normalmente vuelve al remitente del sobre (`MAIL FROM`, o dirección derivada por el sistema); una respuesta normal del usuario se dirige a `Reply-To` si está presente. El nombre mostrado en `From:` no prueba el servidor de origen. **Conclusión:** hay una diferencia que requiere contexto, pero no es automáticamente phishing. [CÁTEDRA: T06, pp. 4–6 y 10–14]

3.5.3 Caso 3. ¿Es pertinente y cuál es su prioridad?

Se denuncia que una PC ajena intentó conectarse a un servidor institucional y que una base de datos con información personal de la propia organización quedó expuesta en Internet. Para el CSIRT de la institución, el segundo hecho es **claramente perti-**

nente y potencialmente **de prioridad alta** por sensibilidad y exposición; el primero requiere precisar si afectó sus usuarios/servicios o fue causado por recursos de su comunidad. La cátedra exige evaluar **tipo, activos afectados, impacto y repercusión en terceros**. [CÁTEDRA: T07, pp. 9–18]

4 PARTE III · Análisis de Malware

4.1 Alcance y método de la teoría

El bloque de malware trata **identificación, indicadores y análisis estático/dinámico básico**; el análisis avanzado de código máquina se menciona como otro nivel, pero queda **fuera del temario de esta versión**. La pregunta que guía todo el proceso es: «¿qué evidencia concreta tenemos sobre qué hace esta muestra y qué consecuencias tiene para gestionar el incidente?». [CÁTEDRA: T08, pp. 2–8]

Regla operacional: toda ejecución de una muestra no confiable se reserva para laboratorios autorizados y aislados, con posibilidad de restauración. Las herramientas de lectura estática no requieren ejecutarla; las herramientas dinámicas sí y, por tanto, implican un riesgo distinto.

4.2 Fundamentos y análisis estático

4.2.1 Qué se considera malware

Malware es software diseñado o utilizado para causar daños, acceder indebidamente a información, alterar sistemas o realizar operaciones perjudiciales. Puede infectar un equipo, funcionar sin propagarse o constituir un componente de una operación más amplia. La teoría enumera **virus, troyanos, backdoors, spyware, scareware, RAT, gusanos, rootkits, botnets, keyloggers, ransomware y downloaders**. [CÁTEDRA: T08, p. 1]

Categoría	Qué hace o cómo se comporta	Distinción para el examen
Virus	Inserta o asocia código a otro objeto/programa y se replica al activarse	Su mecanismo de propagación difiere del gusano
Gusano (*worm*)	Se propaga principalmente entre sistemas, a menudo por red	Puede dispersarse sin intervención continua del usuario
Troyano	Parece legítimo o atractivo y oculta función maliciosa	Se refiere al engaño, no necesariamente a replicación

Categoría	Qué hace o cómo se comporta	Distinción para el examen
Backdoor	Facilita acceso evitando mecanismos esperados de autenticación	Puede ser instalado por otro malware
Spyware	Recopila datos y actividad sin autorización	Espionaje; puede incluir exfiltración
Keylogger	Registra pulsaciones u otros eventos de entrada	No todo registrador de teclas es malicioso en todos los contextos
RAT	Proporciona administración/control remoto	Hay herramientas remotas legítimas; importa el uso y autorización
Rootkit	Oculta presencia o controla capas privilegiadas	Dificulta observación y remoción
Ransomware	Bloquea, cifra y/o roba datos para extorsionar	Puede combinar cifrado y amenaza de filtración
Downloader	Obtiene otras cargas desde red o un recurso externo	Puede ser la primera etapa de una infección
Scareware	Usa alertas falsas para inducir acciones	Ingeniería social mediante software
Botnet	Conjunto de equipos controlados de forma coordinada	«Botnet» es la red, no una única función del binario

Ejemplo. Una falsa actualización instala un downloader; este obtiene una RAT, crea persistencia y conecta con un servidor de control. Puede haber distintas técnicas en una misma infección. Por eso una clasificación no describe por completo sus capacidades.

4.2.2 Para qué se analiza una muestra

Un CSIRT intenta contestar: **¿es malicioso?, ¿qué hace?, ¿qué archivos intervienen?, ¿cómo se elimina?, ¿cómo encontrar otras máquinas afectadas?, ¿qué tráfico permite detectarlo?**. La investigación combina **análisis estático**, que estudia la muestra sin ejecutarla, y **análisis dinámico**, que la observa en ejecución controlada. [CÁTEDRA: T08, pp. 2–6]

La presentación ubica el análisis estático y dinámico básico dentro del trabajo CSIRT; los enfoques avanzados quedan fuera de este temario. La diferencia no es absoluta: observar importaciones y strings puede bastar para una IOC, mientras que un binario ofuscado puede requerir depuración detallada.

4.2.3 Indicadores de compromiso (IOC)

Un **IOC** (*Indicator of Compromise*) es una señal técnica que puede contribuir a identificar sistemas afectados o actividad relacionada. T08 menciona **hashes, archivos, procesos, servicios, claves del registro, IP, URL y consultas DNS**. [CÁTEDRA: T08, p. 4]

Indicador	Ejemplo didáctico	Limitaciones
SHA-256	Digest de un ejecutable	Cambia si el archivo cambia un byte
Archivo creado	`C:\Users\usuario\AppData\...\archivo.exe`	Rutas y nombres pueden variar
Proceso	Nombre, ruta, hash, padre	Un nombre normal puede ser suplantado
Servicio / arranque	Registro de un servicio o Run key	Debe compararse con configuración legítima
Dominio / URL	`https://control.example.test/ping`	Infraestructura puede rotar o ser compartida
IP / DNS	IP de conexión, resolución concreta	Una IP de nube puede alojar múltiples servicios

Ampliación — IOC vs comportamiento. Un hash puntual es un indicador muy específico, pero fácil de evadir alterando un archivo. Una observación como «proceso no firmado que crea claves de arranque y establece conexiones anómalas» es un patrón conductual útil aunque cambie el hash. Se obtiene mayor confianza combinando evidencias.

4.2.4 Flujo de análisis estático básico

La teoría recomienda comenzar por identificación con **antivirus/VirusTotal**, hashes, **strings**, estructura del ejecutable e importaciones. El objetivo es aprender todo lo posible **sin ponerlo en funcionamiento**. [CÁTEDRA: T08, pp. 7–8]

Procedimiento propuesto:

1. Obtener la muestra sin ejecutarla. Registrar origen, tamaño, nombre, fecha, permisos y cadena de custodia.
2. Calcular hashes (al menos SHA-256) para reconocer la muestra en análisis posteriores.
3. Identificar el **tipo real de archivo** por su encabezado y estructura, no solo por extensión.

4. Revisar reputación por hash, cuando sea admisible, evitando subir material sensible a servicios públicos.
5. Extraer cadenas con ASCII/UTF-8/UTF-16 y filtrar rutas, URLs, IP, mensajes y nombres de funciones.
6. Inspeccionar secciones, importaciones/exportaciones, recursos y rasgos de empaquetado.
7. Formular **hipótesis** verificables: descarga, persistencia, exfiltración, cifrado o evasión.
8. Diseñar un análisis dinámico aislado si quedan preguntas críticas.

Ejemplo básico en Linux:

```
file muestra.bin
sha256sum muestra.bin
xxd -l 64 muestra.bin
strings -a -n 6 muestra.bin | head -n 40
strings -a -el muestra.bin | head -n 40
```

`file` intenta reconocer firmas y estructura; `xxd -l 64` muestra los primeros 64 bytes; `strings -a -n 6` busca secuencias imprimibles de longitud seis o más; `-el` pide al programa GNU `strings` una interpretación de texto UTF-16LE. **Estos comandos son de lectura**, no ejecutan el programa analizado. La compatibilidad de opciones depende de la implementación instalada.

4.2.5 Antivirus y VirusTotal

La teoría propone usar **múltiples motores** porque las firmas y heurísticas difieren. **VirusTotal** es un portal de agregación que muestra resultados de motores y datos relacionados: un informe con muchos positivos puede orientar la clasificación; resultados negativos **no demuestran inocuidad** y pueden existir falsos positivos. [CÁTEDRA: T08, pp. 9–12]

Ejemplo de interpretación. Un archivo es identificado por 30 motores como «Trojan». Eso aumenta sospecha, pero no reemplaza estudiar procedencia y conducta. Si solo uno lo marca, puede tratarse de una falsa alarma, de una amenaza nueva o de una

heurística particular; hay que investigar. **No subir binarios propios, documentos con datos personales ni secretos corporativos** a un servicio de análisis público sin autorización.

4.2.6 Strings, ASCII y UTF-16 en Windows

Las **cadenas legibles** incrustadas en un binario pueden revelar rutas, dominios, errores, claves de registro, parámetros o bibliotecas usadas. No obstante, pueden ser señuelos y las strings cifradas, comprimidas u ofuscadas no se ven directamente. [CÁTEDRA: T08, pp. 13–16]

Windows frecuentemente expone interfaces de texto en **UTF-16LE**; de ahí que una palabra parezca intercalada con bytes `00` al verla en hexadecimal. Por ejemplo, `A` se almacena en UTF-16LE como `41 00`, mientras que su codificación ASCII/UTF-8 sería `41`. **Ampliación:** hablar de «2 bytes por carácter» es una simplificación: algunos caracteres Unicode requieren **pares sustitutos**, cuatro bytes en UTF-16.

Caso. Un binario contiene `https://update.example.test/next` como texto visible. Eso es una **hipótesis de destino de red**, no una prueba de que la URL se use ni de que el tráfico sea malicioso. La evidencia mejora si análisis dinámico observa una conexión correspondiente.

4.2.7 Ejecutables de Windows: PE y sus componentes

PE (*Portable Executable*) es un formato utilizado por ejecutables y bibliotecas de Windows (por ejemplo `.exe` y `.dll`). Contiene cabeceras, **secciones**, entradas relativas al cargador, importaciones y exportaciones. La teoría destaca secciones como `.text`, `.rdata`, `.data` y `.rsrc`. [CÁTEDRA: T08, pp. 17–18]

Elemento	Qué significa normalmente	Por qué es útil
`MZ`	Firma inicial DOS heredada del PE	Primer indicio de ejecutable PE, no garantía suficiente
`PE\0\0`	Firma PE señalada por la cabecera DOS	Confirma estructura PE al validar offsets
`.text`	Instrucciones ejecutables	Desensamblar código
`.rdata`	Datos de solo lectura y tablas, según compilación	Strings, importaciones y datos constantes
`.data`	Datos inicializados modificables	Estado global, configuraciones

Elemento	Qué significa normalmente	Por qué es útil
`.rsrc`	Recursos	Iconos, diálogos, traducciones, datos embebidos
Import table	DLL y funciones importadas	Sugiere capacidades
Export table	Funciones expuestas por el módulo	Relevante en DLL

Ampliación — precisión técnica. Las secciones **no tienen nombres obligatorios**; la ubicación real de las tablas se determina por cabeceras/directorios del formato. Un PE puede ser válido sin usar los nombres tradicionales. La cabecera DOS comienza con `MZ`, pero una coincidencia en bytes aislados no prueba que todo el archivo sea ejecutable.

En T08 se señalan funciones Windows que ayudan a inferir intenciones. [CÁTEDRA: T08, pp. 19–20]

API (nombre habitual)	Función general	Pregunta útil
`VirtualProtect`	Cambia permisos de protección de memoria	¿Se convierte memoria en ejecutable?
`LoadLibrary`	Carga una biblioteca	¿Cambia el conjunto de dependencias en ejecución?
`GetProcAddress`	Localiza una función exportada	¿Resuelve APIs dinámicamente?
`CreateService` / `CreateServiceW`	Crea configuración de servicio	¿Intenta establecer persistencia o servicio legítimo?
`InternetOpen`	Abre contexto WinINet	¿Se inicializa cliente HTTP/Internet?
`InternetOpenUrl`	Abre un recurso remoto	¿Con qué URL y en qué circunstancias?
`URLDownloadToFile`	Descarga una URL a archivo	¿Qué se descarga y a dónde?

Precaución: importar `VirtualProtect` o `LoadLibrary` no es demostración de conducta maliciosa. Compiladores, instaladores y software legítimo pueden usarlas. Además, algunas muestras resuelven APIs durante la ejecución y **no aparecen** en la tabla de importaciones.

4.2.8 Recursos, PEStudio y empaquetadores

La sección de **recursos** puede incluir iconos, menús, manifiestos, strings o archivos empaquetados. La teoría usa **Resource Hacker** para explorar recursos y **PEStudio** como herramienta integradora para inspeccionar características estáticas, junto a VirusTotal. [CÁTEDRA: T08, pp. 21–26]

Packing / empaquetado reordena, comprime o cifra el ejecutable, añadiendo un *stub* que recupera el código al ejecutarse. **Ofuscación** transforma representaciones para dificultar comprensión. Entre indicios mencionados figuran **pocas strings**, **pocas importaciones**, presencia de funciones de carga dinámica y **diferencias entre tamaño de archivo (raw size) y tamaño virtual de secciones (virtual size)**. [CÁTEDRA: T08, pp. 27–33]

Importante: un empaquetador no convierte automáticamente el programa en malware: se usa también en herramientas legítimas, instaladores y productos comerciales. La exposición del curso presenta el empaquetado como **señal de investigación**, no como prueba concluyente. Las diapositivas comparan binarios empaquetados y sin empaquetar en sus listas de importaciones y secciones.

Ejercicio resuelto. Una muestra importa solamente `LoadLibrary` y `GetProcAddress`, tiene strings mínimas y sección con alta diferencia entre *raw* y *virtual size*. Hipótesis razonable: **podría** estar empaquetada. Próximos pasos: inspeccionar entropía y punto de entrada; planificar observación del desempaquetado en una VM. No se puede deducir con esos datos quién la creó ni su objetivo.

4.3 Análisis dinámico y persistencia

4.3.1 Qué añade el análisis dinámico

El **análisis dinámico** observa una muestra **mientras se ejecuta**: cambios en archivos, **Registro de Windows**, procesos, servicios, conexiones y datos transferidos. La teoría propone comparar un estado previo del sistema con otro posterior. Pero advierte que una única ejecución no recorre necesariamente **todos los caminos de ejecución**; el malware puede necesitar argumentos, eventos, red, claves, tiempo o interacción específica. [CÁTEDRA: T08, pp. 34–36]

Ejemplo. Un programa sospechoso no crea archivos en la primera prueba. Eso no demuestra que carezca de efectos: quizá espera permisos elevados, una fecha, una instrucción externa o un

nombre concreto de archivo. El resultado correcto es «en este entorno y durante esta ventana de observación no se registró ese cambio».

4.3.2 El laboratorio de análisis

La presentación insiste en utilizar **equipos dedicados o máquinas virtuales**, conservar **snapshots limpios**, impedir que el malware alcance la red real, evitar carpetas compartidas o memorias USB expuestas y **restaurar el estado inicial antes de cada ensayo**. Cita expresamente VirtualBox y el riesgo de propagación ilustrado por **WannaCry**. [CÁTEDRA: T08, pp. 37–40]

Protocolo seguro de trabajo: mantener la VM de análisis **aislada** (sin puente hacia la red de la organización); deshabilitar portapapeles, carpetas compartidas, drag-and-drop y dispositivos externos cuando no sean imprescindibles; configurar una red simulada si se necesita registrar actividad; obtener una instantánea limpia; capturar estado inicial; ejecutar solo muestras autorizadas; registrar cambios; restaurar snapshot.

Sobre privilegios: la diapositiva propone usar permisos de administrador para observar todo el potencial de una muestra. Eso **incrementa el riesgo** y solo tiene sentido en un laboratorio expresamente diseñado para tal nivel de acceso, nunca en el equipo personal o de trabajo.

4.3.3 Registro de Windows: estructura y persistencia

El **Registro de Windows** es una base jerárquica de configuración. T08 destaca que el software puede ejecutarse automáticamente al iniciar el sistema o iniciar sesión aprovechando claves de arranque, y contrasta **HKCU** (perfil del usuario) y **HKLM** (configuración del equipo). [CÁTEDRA: T08, pp. 41–47]

Ubicación	Concepto	Privilegio típico
`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`	Ejecutar aplicaciones al iniciar sesión del usuario actual	Escriturable por ese usuario en su perfil
`HKLM\Software\Microsoft\Windows\CurrentVersion\Run`	Ejecutar al iniciar sesión de usuarios en el equipo	Generalmente requiere privilegios administrativos
`HKLM\System\CurrentControlSet\Services`	Configuración de servicios y controladores	Cambios usualmente privilegiados

Persistencia significa que el código consigue volver a ejecutarse después de finalizar un proceso, cerrar sesión o reiniciar el equipo, según la técnica. Un ejemplo de T08 guarda el binario en el disco y crea una referencia en una **Run key**. Esta técnica está catalogada por **MITRE ATT&CK como T1547.001: Registry Run Keys / Startup Folder**; un servicio malicioso cae en una familia diferente, asociada a la creación o modificación de servicios. [CÁTEDRA: T08, pp. 43–46; AMPLIACIÓN: MITRE ATT&CK]

Qué investigar sin modificar el equipo: registrar ruta, nombre del valor, datos, permisos, fecha de cambio y archivo objetivo; comprobar su firma/hash; contrastar con procesos observados y documentación; revisar si el ejecutable continúa disponible; considerar otras ubicaciones de persistencia antes de concluir erradicación.

Ejemplo de registro de evidencia:

```
Hallazgo: valor en HKCU\...\Run
Nombre: ActualizadorLab
Dato: C:\Users\Usuario\AppData\Local\Lab\actualizador.exe
Proceso observado: actualizador.exe
SHA-256: <digest calculado durante el análisis>
Interpretación: posible autoinicio en sesión del usuario
```

No se muestra un comando de instalación de persistencia, porque el objetivo aquí es reconocerla y documentarla.

4.3.4 Herramientas de registro, procesos y filesystem

Regshot captura y compara estados del Registro. **Autoruns** identifica ubicaciones comunes de autoinicio. **Process Monitor (Procmon)** registra eventos de procesos y threads, filesystem, Registro y parte de la actividad de red; se puede filtrar por ejecutable, PID, ruta u operación. **Process Explorer** ofrece detalle de procesos, DLL cargadas, árbol de procesos y otros datos; la teoría menciona además **Process Hacker** como explorador alternativo. [CÁTEDRA: T08, pp. 48–55 y 57]

Ejemplo de análisis con Regshot: se toma una captura limpia; se ejecuta una muestra autorizada; se realiza la segunda captura; se revisan claves creadas, eliminadas y modificadas. Conviene descartar ruido normal del sistema y correlacionar claves nuevas con hechos observados en Procmon.

Ejemplo con Procmon: se filtra por `Process Name = muestra.exe` y se examinan operaciones que apuntan a `CurrentVersion\Run`, nuevos archivos en perfiles y procesos hijos. Es importante no perder los eventos anteriores a la configuración de los filtros; puede ser útil iniciar captura antes de ejecutar la muestra.

4.3.5 Comunicaciones: Wireshark, tcpdump y servicios simulados

T08 menciona **Wireshark** y **tcpdump** para capturar paquetes, **FakeNet** para simular múltiples servicios de Internet y **FakeDNS** para simular resolución DNS en un laboratorio. Estas herramientas permiten correlacionar URLs y dominios vistos estáticamente con los que se consultan realmente en ejecución. [CÁTEDRA: T08, p. 56]

```
## Ejemplos de inspección en un laboratorio con tráfico propio
sudo tcpdump -i any -nn 'port 53 or port 80 or port 443'
## Leer una captura previamente autorizada
Wireshark captura-lab.pcapng
```

Preguntas observables: ¿qué nombre se resuelve?, ¿qué IP recibe la conexión?, ¿qué puerto y protocolo se usan?, ¿se transmite información identificable?, ¿hay periodos repetitivos que sugieran *beaconing*?, ¿se descargan cargas adicionales? El uso de HTTPS limita la visibilidad del **contenido** desde una captura pasiva, pero no necesariamente de todos los metadatos.

4.3.6 Miniestudio de caso: investigación reproducible

Situación ficticia: un ejecutable adjunto a un correo parece un actualizador. Sin abrirlo se obtiene SHA-256, formato PE, strings que contienen `https://updates.example.test/agent`, e importaciones de WinINet y carga de DLL. Esto **no alcanza** para determinar su finalidad.

En la VM aislada se capturan Regshot y Procmon. Se observa un proceso nuevo, la creación de un valor bajo `HKCU\...\Run` y una petición a un servidor simulado de laboratorio. Se registra que la ruta apunta al ejecutable original. Con esto sí puede afirmarse, en ese ensayo, que **intentó persistir al iniciar sesión y establecer comunicación**. La organización podrá buscar hashes, claves y dominios en otros equipos, **sin olvidar que esas evidencias admiten explicaciones alternativas** si no se correlacionan.

4.3.7 Checklist de seguridad y límites del análisis

- Mantener la muestra sin ejecutar hasta aislarla; nunca asumir que un doble clic es seguro.
- Evitar exposición de secretos a sandboxes públicos.
- Registrar línea temporal y fuente de cada IOC.
- No basar conclusiones en una sola detección antivirus, string o API importada.
- Recordar que VM, emulación y falta de conectividad pueden cambiar la conducta observada.
- No utilizar la red organizacional ni carpetas compartidas con muestras activas.
- Al terminar, restaurar el entorno limpio y guardar una copia de las evidencias fuera del sistema contaminado.

4.4 Comparación final: qué conclusiones permite cada evidencia

Hallazgo	Hipótesis razonable	Qué falta para concluir
String `https://updates.example.test/x`	Puede emplear esa URL	Observar referencia o conexión real
Import `URLDownloadToFile`	Tiene capacidad de descarga	Confirmar la invocación y qué se descargó
VirusTotal: varios positivos	Archivo sospechoso	Evaluar falsos positivos, origen y otras evidencias
Valor Run bajo HKCU que apunta al ejecutable	Autoinicio al iniciar sesión del usuario	Vincular cambio temporal y proceso responsable
Procmon registra creación de archivo	Hubo modificación en ese ensayo	Entender finalidad e impacto
DNS y tráfico observados	Hubo comunicación en el laboratorio	Correlacionar con el proceso y contenido/contexto

La diferencia entre un **indicio**, una **observación** y una **conclusión respaldada** es esencial. Que una API pueda descargar no demuestra que se haya descargado algo; que se vea una clave Run nueva **sí** permite describir ese cambio en el entorno probado, siempre que su procedencia se haya correlacionado. [CÁTEDRA: T08, pp. 19–20, 40–56]

5 Integración y preparación para evaluaciones

5.1 Casos integradores resueltos

Estos cuatro casos relacionan los contenidos de las tres unidades sin exigir reversing ni ensamblador.

5.2 Caso A. Un correo sospechoso que anuncia una deuda

Contexto: la organización recibe una denuncia de un correo con enlace a una supuesta actualización de pagos. Hay discrepancia entre `From` y `Reply-To`. Se recibe también el archivo original `.eml`.

Resolución: (1) evaluar si el destinatario pertenece a la comunidad objetivo; (2) conservar el `.eml` y revisar `Received` desde el primer salto confiable; (3) verificar fechas y zonas horarias; (4) comparar dominio visible con remitente de sobre y, si aparecen, resultados SPF/DKIM/DMARC; (5) inspeccionar la URL como texto, sin abrirla desde un equipo productivo; (6) consultar WHOIS/RDAP para identificar registro de dominio e IP; (7) registrar IOCs y notificar a los contactos apropiados.

Conclusión defendible: una discrepancia entre remitentes más una URL ajena y un pedido de credenciales hacen **probable** un intento de suplantación. El registro RDAP indica quién administra un recurso, pero no demuestra quién redactó el mensaje. [CÁTEDRA: T05–T07]

5.3 Caso B. Software descargado que parece un instalador

Contexto: un ejecutable PE tiene pocas strings, importa `GetProcAddress` y hay alerta de antivirus.

Resolución: (1) calcular SHA-256 y conservar el archivo original sin abrir; (2) validar cabeceras PE y secciones; (3) registrar heurísticas de reputación sin tomarlas como verdades absolutas; (4)

evaluar si hay indicios de packing; (5) preparar VM completamente aislada, snapshot y registros iniciales; (6) observar Procmon, Regshot, Autoruns y tráfico simulado; (7) correlacionar proceso, archivos, rutas de autoinicio y conexión; (8) producir un informe con IOCs y limitaciones.

Conclusión defendible: si se observa una **Run key creada** que apunta a la muestra y una conexión saliente, podemos describir **autoinicio y tráfico en esa ejecución**. No se puede asegurar, sin más pruebas, que el malware siempre actúe igual ni que se haya identificado toda su funcionalidad. [CÁTEDRA: T08]

5.4 Caso D. Distinguir integridad, cifrado y autenticidad

Una organización publica una imagen ISO, su SHA-256 y una firma digital. Un estudiante obtiene el archivo y comprueba su hash. ¿Es lo mismo que verificar la firma? **No.** Si una persona modifica la ISO y también el hash mostrado por una página comprometida, ambos pueden coincidir aunque el artefacto sea malicioso. La firma verificada con una **clave pública autenticada por un mecanismo confiable** permite añadir evidencia de procedencia. Tampoco significa que el software esté libre de vulnerabilidades.

5.5 Caso E. Dirección IP y clasificación de riesgo

Se registran 200 intentos fallidos por minuto desde `192.0.2.15` hacia un servidor de laboratorio y, al mismo tiempo, se observa un error de autenticación en otro equipo. ¿Hay necesariamente un botnet? **No.** Se conoce únicamente la actividad observada. Se debe diferenciar **indicadores** (frecuencia y origen), **hipótesis** (automatización) y **conclusiones probadas**. Los reportes a terceros necesitan tiempo, zona horaria, servicios y evidencia apropiada; la respuesta puede incluir mitigaciones y búsqueda de otros eventos. [CÁTEDRA: T05 y T07]

5.6 Preguntas de autoevaluación con respuestas

Las preguntas son de elaboración didáctica, no un parcial oficial. Se agrupan por los tres ejes y se respondieron en forma breve para facilitar repasos. Para justificar una respuesta, consultar el apartado de desarrollo y la teoría indicada.

5.6.1 Criptografía

1. ¿Base64 brinda confidencialidad? No: cualquier persona puede decodificarlo sin una clave secreta.

2. ¿Qué representa ASCII `0x41`? `A` mayúscula, decimal 65.

3. ¿Por qué ROT13 es reversible sin contraseña? Su desplazamiento es público y fijo; aplicar ROT13 dos veces devuelve el original.

4. ¿Qué es un hash? Transformación determinística de entrada arbitraria a digest de longitud fija.

5. ¿Qué diferencia hay entre preimagen y colisión? Preimagen busca un mensaje para un hash dado; colisión busca dos mensajes diferentes con el mismo digest.

6. ¿Por qué MD5 no sirve para integridad adversarial fuerte? Se conocen ataques prácticos de colisión que permiten construir contenidos distintos con igual digest.

7. ¿Un hash de archivo equivale a firma digital? No: el hash identifica contenido, la firma puede aportar prueba verificable de autoría bajo un esquema de confianza.

8. ¿Por qué usar sal y un algoritmo costoso para contraseñas? Para resistir tablas precalculadas y elevar el costo de probar candidatos.

9. ¿Qué comparten emisor y receptor en cifrado simétrico? Una clave secreta o secreto derivado de manera coordinada.

10. ¿Qué debilidad estructural tiene ECB? Bloques de texto iguales generan bloques cifrados iguales con la misma clave.

11. ¿Qué hace el IV en CBC? Evita que el primer bloque dependa solamente de la clave y el primer bloque de texto plano, mediante XOR previo.

12. ¿Por qué un cifrado autenticado es preferible a CBC sin MAC? Porque detecta modificaciones además de ocultar el contenido.

13. ¿Por qué no se debería usar RC4 en un sistema nuevo? Tiene debilidades conocidas y está obsoleto frente a opciones modernas.

14. ¿En qué se diferencian una sustitución y una transposición? La sustitución cambia los símbolos; la transposición conserva los símbolos y altera su orden.

15. ¿Qué ejemplos históricos menciona la teoría? ROT/César, escítala, transposición columnar y Enigma.

16. ¿Qué es el texto plano y qué es el ciphertext? El primero es el mensaje legible antes del cifrado; el segundo, el resultado cifrado.

17. ¿Por qué una clave no debe reutilizarse sin control entre muchas personas? Porque es más difícil revocar, atribuir acciones y limitar la exposición cuando se compromete el secreto compartido.

18. ¿Cómo difieren los cifrados en bloque y los de flujo? Los primeros transforman bloques de tamaño fijo; los de flujo combinan bytes/bits con una secuencia cifrante.

19. ¿Qué dificultad fundamental tiene el cifrado simétrico? Distribuir, proteger y administrar una clave compartida.

20. ¿Por qué el mismo digest no implica necesariamente el mismo archivo? Las colisiones existen y algunas funciones, como MD5, admiten colisiones construibles.

5.6.2 CSIRT

21. ¿Qué hace un RIR? Administra asignaciones y registros de recursos numéricos de Internet en su región.

22. ¿Qué diferencia WHOIS de RDAP? WHOIS ofrece típicamente texto libre; RDAP entrega respuestas estructuradas JSON mediante HTTP(S).

23. ¿Qué es un bloque ERX? Un rango vinculado a asignaciones históricas anteriores a las actuales estructuras de RIR.

24. ¿Registrar y registrant son sinónimos? No. Registrar es la entidad que gestiona la registración; registrant es el titular.

25. ¿La titularidad WHOIS de una IP identifica necesariamente al atacante? No: suele indicar el administrador del recurso, no al usuario individual.

26. ¿Para qué sirve DNS MX? Para localizar servidores indicados para recibir correo de un dominio y sus preferencias.

27. ¿Qué función cumple SMTP? Transferir mensajes de correo entre agentes y servidores de envío.

28. ¿Qué diferencia existe entre envelope MAIL FROM y From:? El primero forma parte de la transferencia SMTP; el segundo es una cabecera informativa del mensaje.

29. ¿Cómo se leen los Received? Normalmente desde abajo hacia arriba, identificando qué saltos son confiables.

30. ¿Por qué no basta una captura del cuerpo para investigar phishing? Porque faltan trazas, remitentes de transporte, tiempos y otra evidencia técnica.

31. ¿Qué se entiende por comunidad objetivo de un CSIRT? Usuarios, servicios y recursos a los que el equipo brinda respuesta, más incidentes originados desde sus recursos que afecten a terceros.

32. ¿Qué datos no deberían faltar en una notificación? Descripción verificable, origen/destino relevantes, evidencia, fechas con zona horaria y contacto apropiado.

33. ¿Qué factores cambian la prioridad de un incidente? Tipo, sensibilidad de activos, alcance, continuidad, propagación e impacto a terceros.

34. ¿TLP cifra información? No. Indica restricciones de distribución; el cifrado requiere otra tecnología.

35. ¿Cuál es la diferencia entre un CSIRT reactivo y proactivo? El reactivo trata incidentes presentes; el proactivo mejora prevención y detección antes de incidentes.

36. ¿Qué significan ARIN, APNIC, LACNIC, RIPE NCC y AFRINIC? Son los cinco RIR; administran recursos numéricos de Internet en sus regiones.

37. ¿Qué es el contacto de abuso de una asignación IP? Un contacto designado para recibir reportes de uso indebido asociado al recurso.

38. ¿Qué es un recurso legacy o ERX? Un rango asignado históricamente antes de la organización actual de los RIR; puede necesitar una consulta a otra base.

39. ¿En qué se diferencian registry, registrar y registrant? Registro de un TLD, entidad registradora y titular del nombre, respectivamente.

40. ¿Qué diferencia hay entre un registro DNS MX y un registro PTR? MX identifica servidores receptores de correo de un dominio; PTR asocia una IP con un nombre en resolución inversa.

41. ¿Por qué debe indicarse la zona horaria en un reporte? Para correlacionar eventos de sistemas que registran horas locales distintas.

42. ¿TLP sustituye al cifrado con PGP? No. TLP define límites para compartir; PGP protege criptográficamente el intercambio.

5.6.3 Malware

43. ¿Análisis estático ejecuta la muestra? No. Analiza archivo y su estructura sin ponerlo a funcionar.

44. ¿Qué aporta el análisis dinámico? Comportamiento observado en procesos, archivos, registro y comunicaciones durante una ejecución controlada.

45. ¿Qué es un IOC? Señal técnica asociable a compromiso, como hash, dominio o modificación de un registro.

46. ¿Por qué VirusTotal negativo no garantiza seguridad? La detección es incompleta y puede no reconocer amenazas nuevas o modificadas.

47. ¿Qué es un PE? Formato ejecutable utilizado por Windows para ejecutables y DLL, entre otros módulos.

48. ¿Qué sugiere `URLDownloadToFile`? Capacidad potencial de descargar recursos, no evidencia suficiente de que efectivamente se use.

49. ¿Qué indican pocas strings y pocas APIs importadas? Pueden sugerir packing u ofuscación, pero no prueban malware.

50. ¿Para qué sirven Regshot, Autoruns y Procmon? Comparar Registro, listar autoinicios y registrar actividad de procesos y sistema, respectivamente.

51. ¿Qué hace una Run key? Puede configurar la ejecución de un programa al iniciarse una sesión del usuario o del sistema, según su ubicación.

52. ¿Qué representa MITRE ATT&CK T1547.001? Persistencia o ejecución al inicio mediante Run keys o carpetas de inicio.

53. ¿Por qué un snapshot es fundamental? Permite restaurar un estado inicial comparable y reducir contaminación de ensayos sucesivos.

54. ¿Qué es un packer? Un mecanismo que transforma y empaqueta un ejecutable, y a menudo recupera su contenido al ejecutarse; puede dificultar análisis estático, pero no prueba intención maliciosa.

55. ¿Qué secciones PE destaca la cátedra? `.text`, `.rdata`, `.data` y `.rsrc`.

56. ¿Qué permite inferir una importación de `CreateService`? Que el programa dispone de una API relacionada con servicios; no prueba que haya creado uno.

57. ¿Cuál es la diferencia entre HKCU y HKLM? HKCU representa configuración del usuario actual; HKLM, configuración del sistema local, normalmente con permisos más estrictos.

58. ¿Qué diferencia hay entre Run keys y servicios de Windows? Una Run key suele iniciar un programa al iniciar sesión; un servicio es administrado por el Service Control Manager según su configuración.

59. ¿Por qué un único ensayo dinámico puede ser incompleto? La muestra puede tener rutas de ejecución alternativas, requerir parámetros, detectar VM o depender de conectividad.

60. ¿Qué componentes de un informe de malware sirven al CSIRT? Identificación y hashes, alcance, comportamientos observados, artefactos, red/registro, IOCs, límites, medidas de contención y fuentes de evidencia.

6 Glosario de términos

Término	Definición para repaso
AEAD	Cifrado autenticado con datos adicionales; detecta alteraciones de datos protegidos
Backdoor	Mecanismo de acceso por vías no previstas o no autorizadas
Base64	Codificación reversible de bytes en caracteres imprimibles
Botnet	Red de sistemas controlados coordinadamente
CBC	Modo de bloques con encadenamiento mediante XOR y bloque anterior
CERT / CSIRT	Equipo dedicado a coordinar incidentes de seguridad
CIA	Confidencialidad, integridad y disponibilidad
Ciphertext	Datos cifrados resultantes de una operación de cifrado
Digest	Valor resumen producido por una función hash
DNS	Base de datos jerárquica distribuida y protocolo de nombres de Internet
Envelope SMTP	Parámetros de transporte de un mensaje, independientes de ciertas cabeceras
IOC	Indicador de compromiso
IV	Vector de inicialización, usado por determinados modos criptográficos
Keystream	Secuencia cifrante combinada con el texto plano
Malware	Software que ejecuta funciones perjudiciales o no autorizadas
MitM	Intercepción o intermediación de una comunicación
MX	Registro DNS que anuncia los servidores receptores de correo
No repudio	Evidencia para impedir la negación plausible de una acción bajo condiciones dadas
Packing	Empaquetado de binarios que transforma su representación y recuperación
PE	Portable Executable, formato habitual en Windows
Phishing	Engaño mediante comunicaciones que simulan un origen confiable
Plaintext	Texto o datos originales antes del cifrado
PGP	Tecnologías criptográficas para cifrado y firma de datos
Prompt injection	Instrucciones adversarias insertadas en contenido de menor confianza consumido por IA
RDAP	Protocolo estructurado de acceso a datos de registro de Internet

Término	Definición para repaso
Registry	Registro de configuración de Windows
RIR	Registro Regional de Internet
Run key	Clave del Registro utilizada para configurar autoinicio
SHA-256	Función hash de la familia SHA-2 con resumen de 256 bits
SMTP	Protocolo de transferencia de correo electrónico
TLP	Protocolo que indica restricciones de distribución de información
VirtualProtect	API Windows para modificar protecciones de páginas de memoria
WHOIS	Consulta histórica de datos de recursos de Internet

7 Bibliografía, alcance y trazabilidad

7.1 Teorías de la cátedra empleadas

- **T04.** *Criptografía*, `04 Cripto parte 1.pdf`, 30 páginas. Incluye las diapositivas introductorias sobre un posible incidente de cadena de suministro, bases, César, hash y cifrado simétrico.
- **T05.** *CSIRT - Gestión de Recursos de Internet*, `05 CSIRT - Recursos de Internet.pdf`, 20 páginas. RIR, WHOIS, RDAP, DNS y actores de registro.
- **T06.** *Aspectos Operativos SMTP y Análisis de Encabezados*, `06 encabezados_smtp.pdf`, 17 páginas, Nicolás Macia.
- **T07.** *Incidentes de Seguridad & CSIRT*, `07 CSIRT - Gestión de incidentes.pdf`, 26 páginas. CIA, vulnerabilidades, incidentes, gestión, TLP y PGP.
- **T08.** *Análisis de Malware*, `08 Analisis_de_malware.pdf`, 57 páginas. Familias, IOC, análisis estático/dinámico, PE, herramientas y persistencia.
- **T03 (apoyo contextual).** *Introducción a la Ciberseguridad: Conceptos básicos*, `03 Conceptos Básicos.pdf`, 19 páginas. Se emplean únicamente definiciones relacionadas directamente con CSIRT y malware.

La reconstrucción es explicativa, no una reproducción imagen a imagen: las capturas de UI y salidas extensas de WHOIS/RDAP se sintetizan y se explican. Se preserva el **temario conceptual** de cada una de las cinco clases específicas. Los datos de capturas históricas y de servicios públicos pueden haber cambiado.

7.2 Referencias externas para profundizar (ampliaciones)

Estas referencias contextualizan nociones de las teorías. Su inclusión **no las convierte en material oficial evaluable**.

- **NIST SP 800-38A**, modos de operación: https://csrc.nist.gov/pubs/sp/800/38/a/final

- **RFC 5322**, encabezados de email: https://www.rfc-editor.org/info/rfc5322/
- **RFC 7480 y RFC 7482**, RDAP: https://www.rfc-editor.org/info/rfc7480/ y https://www.rfc-editor.org/info/rfc7482/
- **FIRST, Traffic Light Protocol (TLP 2.0)**: https://www.first.org/tlp/
- **NIST SP 800-61**, respuesta a incidentes: https://csrc.nist.gov/pubs/sp/800/61/r3/final
- **MITRE ATT&CK, Registry Run Keys/Startup Folder (T1547.001)**: https://attack.mitre.org/techniques/T1547/001/
- **ICANN / IANA**, DNS y registro: https://www.icann.org/ y https://www.iana.org/domains/root/db
- **VirusTotal**, análisis multivendedor: https://www.virustotal.com/
- **MalAPI**, referencia de API de Windows: https://malapi.io/
- **Libro citado en T08:** Sikorski y Honig, *Practical Malware Analysis*, como bibliografía adicional sobre funciones de Windows.

7.3 Comprobación de cobertura

Unidad	Temas de clase que se desarrollan
Criptografía (T04)	Incidente de supply chain como contexto, encoding, ASCII, criptografía, sustitución/transposición, César/ROT, scytale, Enigma, hash, MD5/SHA, colisiones, contraseñas y firma, simétrica, bloques, ECB/CBC/CFB/OFB, flujo, distribución de claves
CSIRT / recursos (T05)	IP v4/v6, IANA/RIR, cinco RIR, WHOIS, RDAP/JSON, bloques ERX, contacto técnico y abuso, DNS y jerarquía, PTR, registry/registrar/registrant, privacidad, CSIRT y LEA
CSIRT / correo (T06)	SMTP y MX, POP/IMAP, envelope y `MAIL FROM`/`RCPT TO`, cabeceras RFC 5322, `Received`, `Return-Path`, `Reply-To`, lectura de trazas, direcciones privadas, sesiones SMTP, suplantación, herramientas de análisis
CSIRT / incidentes (T07)	CIA, amenaza, vulnerabilidad, incidentes, CERT/CSIRT, comunidad objetivo, recepción, pertinencia, clasificación/priorización, notificación con timestamps y zona, tipos de servicios, buenas prácticas, TLP, PGP

Unidad	Temas de clase que se desarrollan
Malware (T08)	Tipologías, objetivos, IOC, estático, AV/VirusTotal, strings/UTF-16, PE, DLL/imports/exports, funciones, recursos, packers, análisis dinámico, snapshots, peligros y límites, registry, Run keys, servicios, Regshot, Autoruns, Procmon, FakeNet, FakeDNS, Wireshark, tcpdump, Procexp/Process Hacker

Criterio de lectura: distinguir siempre **dato observado**, **inferencia razonable** y **hecho demostrado**. Un manual de estudio es útil si permite explicar no solo qué buscar, sino también qué conclusiones quedan injustificadas con la evidencia disponible.